

Monoidal Parallel Parsing

Carter Teplica, May 2025

Introduction

Many problems in computer science can be reduced to parsing a language. For example, a compiler typically includes a lexer, a parser, a code generator, and a series of additional components which perform steps such as type checking and optimization. In many cases, it is desirable to parse a language in parallel to improve performance.

Intuitively, a parallel parser might operate as follows. First, a sequence of symbols is split into small segments. If this is done straightforwardly, e.g., by splitting the sequence into segments of equal length, the segments may not correspond to “meaningful” syntactic units. Second, these units are parsed in parallel. Finally, the units are combined, in a process which might involve additional parsing to handle fragmentary segments.

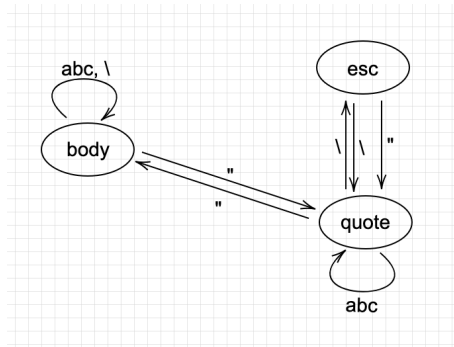
One approach to this process might be to design the outer sequential portion of the parser to carefully choose break points in the input sequence and keep track of how the segments must be reassembled. In this project, we pursue the opposite approach: we consider a fixed and extremely simple outer structure and encapsulate all of the language-specific detail in the inner loop. In particular, we use representations in which we can represent arbitrary segments of the input as single units, and combine them in a way that corresponds to concatenating input segments. In categorical terms, our representations are *monoidal*: that is, they form a set endowed with an associative binary operation (corresponding to concatenation) and a unit (corresponding to the empty string). More precisely, we can think of a program like this from the monoid of strings under concatenation (the “free monoid”) to a monoid of partially-parsed strings.

Over the course of this project, we have constructed several monoidal parsers for simple languages, including a simple XML validator and a program which computes inverses for a 1-dimensional cellular automaton. We have found that our programs often look rather different from traditional parsers, but that they can generally be constructed out of simple monoidal building blocks. While we focus on theory, we find good parallel performance in some preliminary experiments.

Finite automata

Regular languages can be parsed by deterministic finite automata. A DFA with states Q and symbols Σ can be regarded as a set of digraphs over nodes corresponding to Q , with one digraph for each symbol in Σ . Identifying digraphs with their transition matrices T_{σ_n} , the effect on the state of a sequence of symbols $\sigma_1 \dots \sigma_n$ can be expressed as the product $\prod_{i=1, \dots, n} \sigma_i$, which we can compute efficiently using a parallel fold.

For example, we can use this matrix formalism to parse a language of quoted strings. Consider a language in which a sequence is valid if it consists of a mix of unquoted text and quoted string literals, where a string literal is bounded by quotes with the caveat that a backslash-escaped quote does not end the string. We can represent this by this DFA:



If we represent states as vectors where the components correspond to [body quote esc], we can represent the transitions as matrices:

$$\text{abc} \mapsto \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

$$" \mapsto \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

$$\backslash \mapsto \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

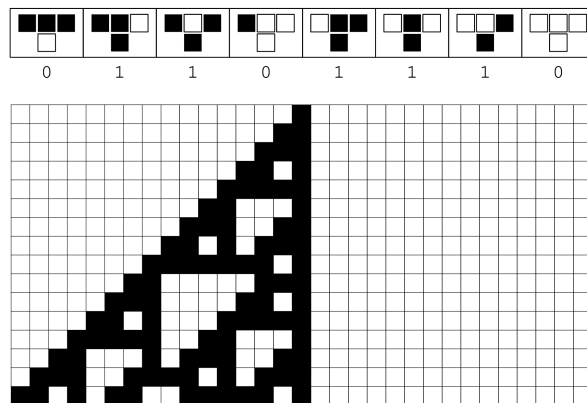
The matrix formalism needs modification to handle more complex languages. For example, the untyped Dyck language can *nearly* be parsed using the rule

$$(\mapsto \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix} \quad) \mapsto \begin{pmatrix} 1 & 0 \\ -1 & 0 \end{pmatrix}$$

with the caveat that this fails to reject strings like $))) (((($.

Inverting a cellular automaton

Rule 110 is a one-dimensional, two-state cellular automaton which notable for being Turing complete. It is defined by the following transition rules,¹ in which the state of a cell at a time t depends on its state and the states of its two neighbors at time $t - 1$:



,

¹Image: "Rule 110". Wolfram MathWorld. Accessed May 2025.

One common problem in the study of cellular automata is to invert the automaton: that is, to find the set of predecessors of a given state. In some cases, this can be a difficult problem. To compute immediate predecessor states of a given Rule 110 state, we can cover the cellular automaton's tape with overlapping blocks of two cells, so that the predecessors of a given cell are determined by the two blocks that overlap it. We can then construct a nondeterministic finite automaton whose edges represent the possible transitions between blocks for each successor state. Writing the states as pairs, these are:

```
0 -> [(00, 00), (10, 00), (11, 11)]
1 -> [(00, 01), (01, 10), (01, 11), (10, 01), (11, 10)]
```

For a monoidal structure, we can combine transitions by taking a union over the middle states. In MaPLe, we can write this with a series of list operations:

```
fun op @@ (l : t, r : t) : t =
  product (l, r) |>
  List.filter (fn ((a, b), (c, d)) => b = c) |>
  List.map (fn ((a, b), (c, d)) => (a, d)) |>
  unique
```

Note that this implementation is inefficient for large numbers of states—each merge takes at least quadratic time in the number of states, and possibly as bad as quartic time with poor choices of primitives. A more efficient implementation might use a sorted array of states, which could be constructed and accessed in $O(n \log(n))$ time.

Two-level parser

As noted above, parsing tools often operate in multiple stages: for example, a typical compiler first lexes the input and then parses a sequence of tokens. To handle tokenization and similar hierarchical structures, we can use a monoid with a “two-level” structure. In categorical terms, this is essentially the composition of a lexer homomorphism (from the free monoid to the monoid of tokens and token-fragments) and a parser homomorphism (from the monoid of tokens and token-fragments to some monoid for higher-level structures). Concretely, we represent a given segment as a triple consisting of a lexed middle section and two unlexed ends. In pseudo-SML:

```
functor TwoLevelM(LexM: MONOID; ParseM: MONOID; val lex : LexM.t -> ParseM.t) =
struct
  datatype t = L of LexM.t
             | LPL of (LexM.t, ParseM.t, LexM.t)

  fun L(a)          @@ L(b)          = L(a @@ b)
    | L(a)          @@ LPL(b, p, c) = LPL(a @@ b, p, c)
    | LPL(a, p, b) @@ L(c)          = LPL(a, p, b @@ c)
    | LPL(a, p, b) @@ LPL(c, q, d) = LPL(a, p @@ lex(b @@ c) @@ q, d)

  id = L(LexM.id)
  break = LPL(LexM.id, ParseM.id, LexM.id)
end
```

When we instantiate `TwoLevelM`, we specify four things:

- a map from characters to lexer states (states `L(a)` of the lexer monad as well as breaks);
- the composition operation for the lexer;
- a map from lexer states to parser states; and
- the composition operation for the parser.

For example, we can use this structure to search a document for its longest line, as follows:

- *a map from characters to lexer states*
 - `Map \n -> break | _ -> 1` in the monad where...
- *the composition operation for the lexer*
 - ...composition is addition...
- *a map from lexer states to parser states*
 - ...then map `i -> i` in the monad where...
- *the composition operation for the parser.*
 - ...composition is `Int.max`.

We can combine DFAs, two-level parsers, and one more monoidal structure to create a validator for a slightly simplified version of XML.

XML validator

Suppose we've already lexed some valid XML and we're looking at a sequence of tags. How can we represent its effect on the rest of the document? If we remove any correctly paired tags, we can represent the remaining tokens as a stack of unmatched closing tags followed by a stack of unmatched opening tags. These can be composed:

```
                                <these><open>
+ </open></these></please> <and><close><these>
    + </these></close> <thank><you>

= </please> <and><thank><you>
```

In pseudo-SML, we can represent each transition as follows:

```
datatype transition = Transition of (tagflavor Seq.t) * (tagflavor Seq.t)
                      (* (closes, opens) *)
                      | TransitionError
```

To compose two transitions, we first “reduce” by pairing off the opening brackets of the first with the closing brackets of the second. If there is a conflict in pairing off the brackets, we return a `TransitionError`. Otherwise, we may have excess opening or closing brackets (though not both); we append these, if any, to the appropriate sequence of brackets and return the new (close, open) pair.

```
fun reduce(opens, closes) =
  let
    val nCommon = Int.min (Seq.length opens, Seq.length closes)
    val openExcess = Seq.drop opens nCommon
    val closeExcess = Seq.drop closes nCommon
    val zippedCommon = (Seq.zipWith
      (op =)
      ((Seq.take opens nCommon),
       (Seq.take closes nCommon)))
    val commonSame = Seq.reduce boolAnd true zippedCommon

  in
    if commonSame then
      SOME (openExcess, closeExcess)
    else
      NONE
  end
```

```

fun compose(left, right) =
  case (left, right) of
    (TransitionError, _) => TransitionError
  | (_, TransitionError) => TransitionError
  | (Transition(leftClose, leftOpen), Transition(rightClose, rightOpen)) =>
    case reduce(leftOpen, rightClose) of
      NONE => TransitionError
    | SOME (openExcess, closeExcess) => Transition(
      Seq.append(leftClose, closeExcess),
      Seq.append(rightOpen, openExcess))

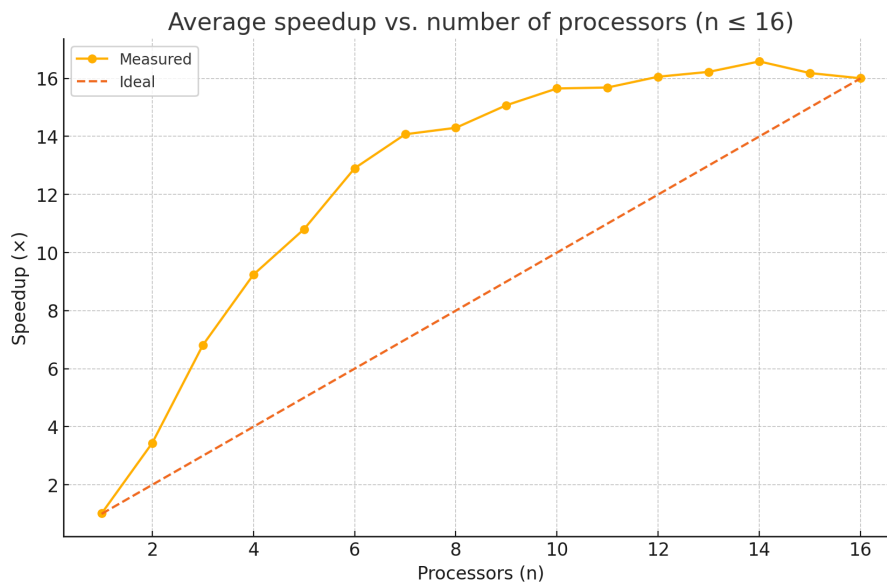
```

(This runs in polylogarithmic span and linearithmic work: in particular, it can be slow when tags are very deeply nested. It might be possible to achieve linear work by using an alternative to the relatively expensive `Seq.append`.)

To turn this into a character-level parser, we wrote a discrete finite automaton which accepts a sequence of characters. Unlike the DFA for Rule 110, this DFA is *weighted*. The weights on the transitions themselves have a monoidal structure, which keeps track of the gender (opening, closing, or self-closing) of the tag as well as the tag's name. (We write this by composing two smaller monoids with a Cartesian product. It's monoids all the way down!) The `lex` map unpacks this information from the graph structure and returns a `Transition` object; finally, these are composed as above to recognize XML.

Performance

We tested our XML validator on a moderately large (8.8 MiB) XML document and observed its performance on a 16-core Intel MacBook Pro.



Our validator was moderately slow in absolute terms, taking about 51 seconds with one thread, but it had strong parallel performance. To our surprise, the speedup factor was consistently greater than the number of threads. We hypothesize that this may be due to cache effects (i.e., possibly the $n = 1$ case is memory-bound and higher numbers of threads are able to take advantage of cache locality).

